

Računalniški Sistemi Homework 4

Andraž Klobučar (63210146)
Filip Gros (63210097)

1 Introduction

In the fourth homework, we analyzed the performance of GPU kernels using the GEM5 GPU simulation environment. The assignment focused on three kernels: histogram computation, matrix transpose, and sparse matrix-vector multiplication. For each kernel, we compared a baseline implementation with an optimized or modified version in order to study how GPU performance is affected by memory contention, shared-memory usage, memory access patterns, and control-flow divergence. The experiments were performed with different numbers of compute units, allowing us to observe how each implementation scales and how architectural behavior influences overall efficiency.

2 Task 1: Analyzing the performance of GPU kernels in GEM5

The evaluation compares two GPU histogram kernel implementations: a naive version using atomic updates to a single global histogram, and an optimized version that reduces contention by using per-workgroup privatization in shared (LDS) memory followed by a final merge. Performance was measured for 2, 4, and 8 compute units (CUs) using hardware counters for memory behavior, compute throughput, and execution time.

Metric	Algorithm	2 CU	4 CU	8 CU
vpc	Naive	3.800	1.947	0.995
totalCycles	Naive	828 386	807 754	789 904
ldsBankAccess	Naive	0	0	0
vALUInsts	Naive	22 528	11 264	5 632
mean loadLatencyDist	Naive	6 150 710.2	12 374 920.4	24 149 654.3
vpc	Optimised	21.647	14.086	7.200
totalCycles	Optimised	230 092	176 800	172 932
ldsBankAccess	Optimised	393 216	196 608	98 304
vALUInsts	Optimised	28 672	14 336	7 168
mean loadLatencyDist	Optimised	805 869.9	1 637 491.8	4 103 802.8

Table 1: Comparison of naive and optimised algorithms across different numbers of compute units (CU).

The naive implementation is memory-bound due to frequent global atomic updates, causing high contention and poor memory efficiency. This is reflected in zero LDS usage ('ldsBankAccess'), low vector utilization ('vpc'), and high cycle counts ('totalCycles'). While increasing CUs reduces per-CU instruction counts, it does not improve scalability, as

global memory contention increases, leading to higher load latency ('mean loadLatencyDist').

The optimized version shifts most updates to shared memory, significantly reducing global memory pressure. This is confirmed by high LDS usage and substantially lower execution cycles $\approx 3-4\times$ reduction. Load latency decreases by an order of magnitude, and vector throughput ('vpc') improves, indicating better compute efficiency and reduced memory serialization.

Overall, the naive kernel remains limited by global memory contention, while the optimized version scales more effectively by decoupling updates from global synchronization, resulting in improved performance across all CU configurations.

3 Task 2

In the second task, we tested the implementation of matrix transpose kernels in order to evaluate the impact of shared memory optimization on memory access efficiency and overall kernel performance.

Metric	Algorithm	2 CU	4 CU	8 CU
vpc	Naive	7.481	3.412	1.472
totalCycles	Naive	525 584	576 176	667 969.625
ldsBankAccess	Naive	0	0	0
vALUInsts	Naive	36 864	18 432	9 216
mean loadLatencyDist	Naive	7 990 418.2	18 857 941.4	42 416 970.7
vpc	Shared	22.026	10.904	5.481
totalCycles	Shared	315 391	318 554	316 848.625
ldsBankAccess	Shared	262 144	131 072	65 536
vALUInsts	Shared	53 248	26 624	13 312
mean loadLatencyDist	Shared	2 520 376.2	5 901 159.9	12 482 820.8

Table 2: Performance comparison of naive and shared memory matrix transpose kernels across different numbers of compute units (CU).

The naive transpose implementation is strongly memory-bound because global memory writes are performed in a strided and non-coalesced manner. Although reads from matrix A are contiguous, writes to matrix B generate inefficient memory transactions, resulting in high global memory latency and poor vector unit utilization. This behavior is reflected in the low vector-per-cycle metric ('vpc') and the very high 'mean loadLatencyDist', which increases exponentially as the number of CUs grows. The increasing 'totalCycles' with higher CU counts further indicates limited scalability

caused by memory-system contention. Since the naive kernel does not use local data share memory, like in previous task the ‘ldsBankAccess’ metric remains zero for all configurations.

The shared-memory implementation improves performance by first loading matrix tiles into LDS memory and then performing the transpose locally before writing results back to global memory. This approach enables coalesced accesses for both reads and writes, substantially reducing global memory traffic and memory latency. Consequently, the shared-memory implementation achieved approximately $2.4\times$, $1.8\times$, and $2.1\times$ lower ‘totalCycles’ for 2, 4, and 8 CU configurations, while also providing approximately $2.9\times$, $3.2\times$, and $3.7\times$ higher ‘vpc’ values compared to the naive implementation. The reduction in ‘mean loadLatencyDist’ demonstrates the effectiveness of LDS as a low-latency staging buffer. The ‘ldsBankAccess’ values confirm active usage of shared memory, while the additional synchronization and LDS operations slightly increase the number of executed vector ALU instructions (‘vALUInsts’). Padding the LDS tile to 16×17 elements successfully minimizes bank conflicts and improves memory parallelism.

Overall, the shared-memory transpose kernel consistently outperformed the naive implementation, confirming that coalesced memory accesses and LDS-based tiling are effective optimizations for reducing memory bottlenecks and improving GPU utilization.

4 Task 3: Analyzing the impact of thread divergence on performance of SpMV kernel

For this task, we were given a program that computes sparse matrix-vector multiplication. We analyzed and compared two kernel implementations: the divergent implementation and the uniform implementation. The divergent implementation launches the kernel in the original order, while the uniform implementation sorts the rows of the matrix in ascending order by the number of nonzero values. We ran both implementations using 2, 4, and 8 compute units.

Version	N_{CU}	μ_{CFD}	σ_{CFD}	Avg. vector ALU instr.	Global reads	Global writes	Coal. global accesses
Div.	2	33.496	19.052	6328	3088	16	3104
Sort.	2	61.170	10.113	3448	1648	16	1664
Div.	4	33.496	19.053	3164	3088	16	3104
Sort.	4	61.135	10.153	1724	1648	16	1664
Div.	8	33.496	19.055	1582	3088	16	3104
Sort.	8	60.976	10.324	862	1648	16	1664

Table 3: Task 3 profiling results. CU denotes compute units, CFD denotes control-flow divergence, and Coal. denotes coalesced.

Table 3 shows the results of our experiments running the two kernels using different numbers of compute units. We can see that the sorted version improves control-flow behaviour for all tested CU counts. For the divergent kernel, the mean control-flow divergence metric is approximately 33.5 active lanes per instruction, meaning that $\approx 52.3\%$ of the wavefront lanes are active on average. After sorting the rows, the mean increases to approximately 61 active lanes, or about 95% lane utilization.

The standard deviation of the control-flow divergence metric also decreases from about 19.05 in the divergent version to about 10.1 in the sorted version. Therefore, the sorted ver-

sion not only has more active lanes on average, but also has more consistent lane activity across instructions.

The average number of vector ALU instructions per CU decreases as the number of CUs increases, since the same total work is distributed across more compute units. For example, the divergent version decreases from 6328 at 2 CUs to 1582 at 8 CUs, while the sorted version decreases from 3448 to 862. For each CU count, the sorted implementation issues fewer vector ALU instructions than the divergent implementation. This is because sorting rows by length reduces the number of inactive lanes within each wavefront. As a result, fewer wavefront-level loop iterations are executed with only a small subset of lanes active.

The sorted version reduces global reads and coalesced global accesses by approximately 46.6%. Global writes remain constant for both versions and all CU counts, which is expected because both kernels produce the same number of output rows.

5 Conclusion

The experiments show that GPU performance depends strongly on memory behavior and control-flow regularity. The optimized histogram kernel reduced global memory contention through LDS privatization, while the shared-memory transpose improved memory coalescing and reduced global memory latency. In the SpMV task, sorting rows by nonzero count reduced wavefront divergence and improved lane utilization.

Overall, the optimized implementations performed better because they addressed the main bottleneck of each workload. The results also show that increasing the number of compute units does not guarantee proportional speedup when memory contention, inefficient accesses, or thread divergence limit hardware utilization.